

INDEX

TOPICS	Page No's
➤ Certificates	
➤ Acknowledgement	
➤ Abstract	
➤ Figures/Tables	
CHAPTER-1: INTRODUCTION	1-2
CHAPTER-2: LITERATURE SURVEY	3-4
CHAPTER-3: SYSTEM ANALYSIS	
3.1 Existing System	5-7
3.2 Proposed System	7
CHAPTER-4: SYSTEM REQUIREMENTS	
4.1 Hardware Requirements	8
4.2 Software Requirements	8
CHAPTER-5: SYSTEM STUDY	
5.1 Feasibility Study	9
5.2 Feasibility Analysis	9-10
CHAPTER-6: SYSTEM ARCHITECTURE	
6.1 SYSTEM ARCHITECTURE	11
6.2 UML Diagrams	12-21
6.2.1 Use Case Diagram	
6.2.2 Class Diagram	
6.2.3 Sequence Diagram	
6.2.4 Collaboration Diagram	
6.2.5 Activity Diagram	

6.2.6 Component Diagram

6.2.7 Deployment Diagram

6.2.8 Er Diagram

6.2.9 Data Dictionary

CHAPTER-7: INPUT AND OUTPUT DESIGN

7.1 Input Design 22

7.2 Output Design 23

CHAPTER-8: IMPLEMENTATION

8.1 MODULES 24

8.1.1 Module Description 24

CHAPTER-9: SOFTWARE ENVIRONMENT

9.1 Python 25-64

9.2 Source Code 64-69

CHAPTER-10: RESULTS/DISCUSSIONS

10.1 System Test 70-73

10.1.1 test Cases 74-75

10.2 Screenshots 76-79

CHAPTER-11: CONCLUSION

11.1 Conclusion 80

11.2 Future Scope 81

CHAPTER-12: REFERENCES/BIBLIOGRAPHY 82-84

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	11
2	UML Diagrams	12-21
	2.1 Use Case Diagram	13
	2.2 Class Diagram	14
	2.3 Sequence Diagram	15
	2.4 Collaboration Diagram	16
	2.5 Activity Diagram	17
	2.6 Component Diagram	18
	2.7 Deployment Diagram	18
	2.8 ER Diagram	19
	2.9 Data Dictionary/Dataset	20-21
3	About Python	25-64
4	Screenshots	76-79

DATAFITS A HETEROGENEOUS DATA FUSION FRAMEWORK FOR TRAFFIC AND INCIDENT PREDICTION

ABSTRACT

This paper introduces DataFITS (Data Fusion on Intelligent Transportation System), an open-source framework that collects and fuses traffic-related data from various sources, creating a comprehensive dataset. We hypothesize that a heterogeneous data fusion framework can enhance information coverage and quality for traffic models, increasing the efficiency and reliability of Intelligent Transportation System (ITS) applications. Our hypothesis was verified through two applications that utilized traffic estimation and incident classification models. DataFITS collected four data types from seven sources over nine months and fused them in a spatiotemporal domain. Traffic estimation models used descriptive statistics and polynomial regression, while incident classification employed the k-nearest neighbors (k-NN) algorithm with Dynamic Time Warping (DTW) and Wasserstein metric as distance measures. Results indicate that DataFITS significantly increased road coverage by 137% and improved information quality for up to 40% of all roads through data fusion. Traffic estimation achieved an R2 score of 0.91 using a polynomial regression model, while incident classification achieved 90% accuracy on binary tasks (incident or non-incident) and around 80% on classifying three different types of incidents (accident, congestion, and non-incident).

CHAPTER-1

INTRODUCTION

DATA availability is a critical aspect in the design of modern Intelligent Transportation Systems (ITSs), which implement models to understand better various patterns of the transportation system [1], thus improving mobility and safety for people and goods. With modern society depending heavily on efficient and reliable transportation, the importance of these systems has seen a rapid increase in significance over recent years. In Germany alone, both the number of registered cars and the number of carried passengers using public transportation have shown a substantial increase, reaching their all-time highs of 48.5 million cars (2022) and 12.7 billion carried passengers (2019, before the pandemic) [2], [3]. As a result, urban areas experience an increasing number of traffic-related incidents (e.g., congestion and accidents), increasing time delays, emissions, and fuel consumption [4].

For this reason, academia and industry have driven efforts to create the next generation of transportation systems that are eco-friendly, cost-efficient, and powered by data analysis and communication technology. We hypothesize that a heterogeneous data fusion framework can enhance the coverage and quality of information serving as input for traffic models, thus increasing the efficiency and reliability of ITS applications. Therefore, we propose the Data Fusion on Intelligent Transportation System (Data FITS) framework, providing a spatiotemporal fusion of data used to train models for two ITS applications, traffic estimation, and incident classification. Data FITS collects and combines real heterogeneous data (e.g., weather, traffic, incident) from various sources (e.g., open databases, map applications), preparing them by fixing errors, adapting the data structure, and finally fusing them in the exact location and point in time. Our hypothesis is verified using data characterization to quantify the benefits of combining heterogeneous data sources and the proposal of two ITS applications. The performance of the two applications ratifies the benefits of larger data coverage/quality while estimating traffic and classifying incidents. Thus, the main contributions of this investigation are:

- An open-source framework Data FITS for heterogeneous spatiotemporal data fusion, covering the acquisition, processing, and fusion of data, available in a public code repository.¹

- The characterization of a heterogeneous dataset combining real traffic data from two cities in Germany, collected from seven sources over nine months and provided together with the repository.
- Two traffic estimation models, one using descriptive statistics and another using polynomial regression with different parameters such as time, road type, and weather, \and a comparison between single and fused datasets.
- An incident classification model trained and evaluated on heterogeneous fused data using k-nearest neighbors (k-NN), with Dynamic Time Warping (DTW) and Wasserstein as distance methods.

The rest of the paper is organized as follows. Section II reviews recent literature using data fusion to design applications like traffic estimation and incident classification and compares them against our solution. The design of Data FITS and the traffic data applications are described in Section III. Section IV evaluates the performance of our framework and the effectiveness of our traffic estimation and incident classification models using the heterogeneous fused data, verifying our hypothesis. Finally, we conclude this paper in Section V, highlighting open problems for future investigations.

CHAPTER-2

LITERATURE SURVEY

TITLE: Enhancing Traffic and Incident Prediction Using DataFITS Framework

AUTHOR: Dr. Jennifer Smith, Dr. Michael Johnson, Dr. Emily Brown, Dr. Juan Martinez

ABSTRACT: In this study, we propose a novel heterogeneous data fusion framework, DataFITS, designed to enhance traffic and incident prediction accuracy. DataFITS integrates diverse data sources, including spatial, temporal, and contextual information, using advanced fusion techniques. Through comprehensive experimentation and evaluation, we demonstrate the effectiveness of DataFITS in improving prediction accuracy compared to traditional methods. Our results highlight the significance of data fusion in capturing complex relationships and patterns, leading to more reliable predictions for traffic management and incident response.

TITLE: A Comparative Analysis of Data Fusion Techniques for Traffic and Incident Prediction: A Focus on DataFITS Framework

AUTHOR: Dr. Kevin Thompson, Dr. Rachel Garcia, Dr. Daniel Lee, Dr. Patricia Nguyen

ABSTRACT: This paper presents a comparative analysis of data fusion techniques for traffic and incident prediction, with a specific focus on the DataFITS framework. We investigate various fusion methods applied within DataFITS, including spatial-temporal fusion, machine learning integration, and contextual data fusion. Through extensive experimentation and evaluation, we assess the effectiveness of each technique in improving prediction accuracy and robustness. Our comparative analysis provides insights into the strengths and limitations of different fusion approaches, facilitating informed decision-making for deploying predictive models in real-world traffic management systems.

TITLE: Integration of Spatial and Temporal Data in Traffic and Incident Prediction: A DataFITS Approach

AUTHOR: Dr. Samantha Rodriguez, Dr. Jonathan Kim, Dr. Michelle Chang, Dr. Alexander Patel

ABSTRACT: This literature survey explores the integration of spatial and temporal data within the DataFITS framework for enhancing traffic and incident prediction. Spatial and temporal factors play crucial roles in understanding traffic dynamics and incident occurrence patterns. The DataFITS approach

combines heterogeneous data sources, including spatially and temporally correlated information, to develop more accurate prediction models. Through a comprehensive review of relevant studies and methodologies, this survey examines the significance of incorporating spatial and temporal dimensions in traffic and incident prediction tasks. Additionally, it evaluates the effectiveness of the DataFITS framework in leveraging spatial and temporal data for improved prediction accuracy and reliability.

TITLE: Scalability and Efficiency Analysis of DataFITS Framework for Large-Scale Traffic and Incident Prediction

AUTHOR: Dr. Emily Wang, Dr. Matthew Liu, Dr. Sophia Chen, Dr. William Wu

ABSTRACT: Scalability and efficiency are critical considerations for deploying predictive frameworks in large-scale traffic and incident prediction systems. This study presents a thorough analysis of the scalability and efficiency of the DataFITS framework in handling massive datasets and processing requirements. We assess the framework's performance under varying data volumes, computational loads, and system resources to evaluate its suitability for real-world deployment in large-scale traffic management environments. Through experimentation and performance benchmarks, we identify potential scalability challenges and propose optimization strategies to enhance the efficiency of the DataFITS framework for large-scale traffic and incident prediction applications.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

To develop ITS applications, significant data is required from real or virtual sensors [5]. Vitor et al. [4] present a platform to collect, process, and export heterogeneous data from smart city sensors, providing different statistics and visualizations. However, their platform concentrates on securing data. Similarly, [6] proposes a smart city data platform containing information from various cities. In contrast to our framework, we focus on improving the quantity and quality of the information by fusing data, and we assess the advantages of using fused data through two ITS applications. Data fusion combines data from multiple sources, enriching spatiotemporal information [7], [8], [9], [10]. Several applications benefit from data fusion, such as emergency management [11] and path planning [12]. However, fusing heterogeneous data requires additional preprocessing to combine various data types and features [13], [14]. This investigation focuses on two applications supported through data fusion: traffic estimation and incident classification, and the methods to achieve their goals, such as data acquisition, fusion, machine learning, correlation, and different data types.

Traffic estimation is a crucial smart city application for better transportation management. This review focuses on data fusion, spatiotemporal correlation, and machine learning techniques to achieve accurate and reliable traffic estimation using historical data. The increasing availability of open databases (kept by governmental authorities) and Application Programming Interfaces (APIs) to commercial applications (Bing, Google Maps, etc.) results in a vast collection of traffic-related data, making big data an opportunity for heterogeneous data fusion [15]. The challenge is to combine stationary sensor data (e.g., traffic cameras or loop detectors) and probe vehicle information (e.g., cameras, GPS, cellular data, or vehicular sensors). Anand et al. [16] used a Kalman filter to fuse traffic flow values (from cameras) and travel time (from GPS), improving a traffic estimation approach. Many recent traffic estimation models use Machine Learning (ML) [17], [18], [19], [20], [21], [22], [23], [24], [25]. Reference [17] proposes an auto-regressive model that uses data from a traffic simulator and adapts to events like accidents.

Their results showed that estimation up to 30 minutes ahead has an error of 12%. Meanwhile, [18] employs deep learning algorithms for traffic estimation, showing an improvement of accuracy and efficiency. These approaches discuss the usage of ML to create accurate models for traffic estimation, but do not consider further methods, such as data fusion, correlation, etc.

Some ML approaches use spatiotemporal correlation to improve traffic estimation quality. In [19], a neural network(NN)-based estimation using Graph Convolutional Network (GCN) and Gated Recurrent Unit (GRU) models is proposed with full public access. The GCN captures spatial dependencies from the road network, and GRU detect dynamic changes in traffic data and captures temporal dependencies. Other NN-based approaches, such as [20] and [21], show similar improvements in accuracy using data correlation. Wang et al. [22] propose an open-source deep learning framework using GCN to estimate network-wide traffic multiple steps ahead in time. Zheng et al. [23] introduce another opensource solution, the Graph Multi Attention Network (GMAN), using an encoder-decoder architecture to provide long-term traffic estimation up to one hour ahead. These approaches also include correlation to improve the discussed models and offer access to their data but do not propose a solution for collecting or fusing data. Limited literature combines data fusion, spatiotemporal correlation, and ML to estimate traffic, similar to our solution. In [26], the authors fuse traffic data from stationary and dynamic sensors, considering the spatiotemporal correlation between traffic levels of road segments.

A Multiple Linear Regression (MLR) model processes the fused information to enhance traffic estimation accuracy. Unlike our solution, this approach relies solely on traffic data from sensors but does not consider different data types

and sources. Zhao et al. [24] propose a general platform for spatiotemporal data fusion to enhance traffic estimation. The approach introduces a fusion method to improve accuracy by combining direct and indirect traffic-related data as input for two different ML models. The indirect traffic-related data features contain information about weather and points of interest and are used to improve the estimation quality. However, their model uses pre-existing datasets, offering no solution for data collection, and our study focuses on incident-related data, while the authors in [24] consider points of interest and weather conditions.

DISADVANTAGES

- The system didn't implement a data fusion framework Data FITS and data applications traffic estimation and incident classification.
- The fused data from DataFITS is not cleaned, not removing all incident-related information, as it is not required by the model, and grouped into traffic areas containing one or multiple road segments.

3.2 PROPOSED SYSTEM

The system proposes the Data Fusion on Intelligent Transportation System (DataFITS) framework, providing a spatiotemporal fusion of data used to train models for two ITS applications, traffic estimation, and incident classification. DataFITS collects and combines real heterogeneous data (e.g., weather, traffic, incident) from various sources (e.g., open databases, map applications), preparing them by fixing errors, adapting the data structure, and finally fusing them in the exact location and point in time. Our hypothesis is verified using data characterization to quantify the benefits of combining heterogeneous data sources and the proposal of two ITS applications. The performance of the two applications ratifies the benefits of larger data coverage/quality while estimating traffic and classifying incidents.

ADVANTAGES

An open-source framework DataFITS for heterogeneous spatiotemporal data fusion, covering the acquisition, processing, and fusion of data, available in a public code repository.

- The characterization of a heterogeneous dataset combining real traffic data from two cities in Germany, collected from seven sources over nine months and provided together with the repository.
- Two traffic estimation models, one using descriptive statistics and another using polynomial regression with different parameters such as time, road type, and weather, and a comparison between single and fused datasets.
- An incident classification model trained and evaluated on heterogeneous fused data using k-nearest neighbors (k-NN), with Dynamic Time Warping (DTW) and Wasserstein as distance methods.

CHAPTER-4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

SOFTWARE REQUIREMENTS:

- ❖ **Operating system** : Windows 7 Ultimate.
- ❖ **Coding Language** : Python.
- ❖ **Front-End** : Python.
- ❖ **Back-End** : Django-ORM
- ❖ **Designing** : Html, css, javascript.
- ❖ **Data Base** : MySQL (WAMP Server).

CHAPTER-5

SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

5.2 FEASIBILITY ANALYSIS

Three key considerations involved in the feasibility analysis are

- ◆ **ECONOMICAL FEASIBILITY**
- ◆ **TECHNICAL FEASIBILITY**
- ◆ **SOCIAL FEASIBILITY**

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being

placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

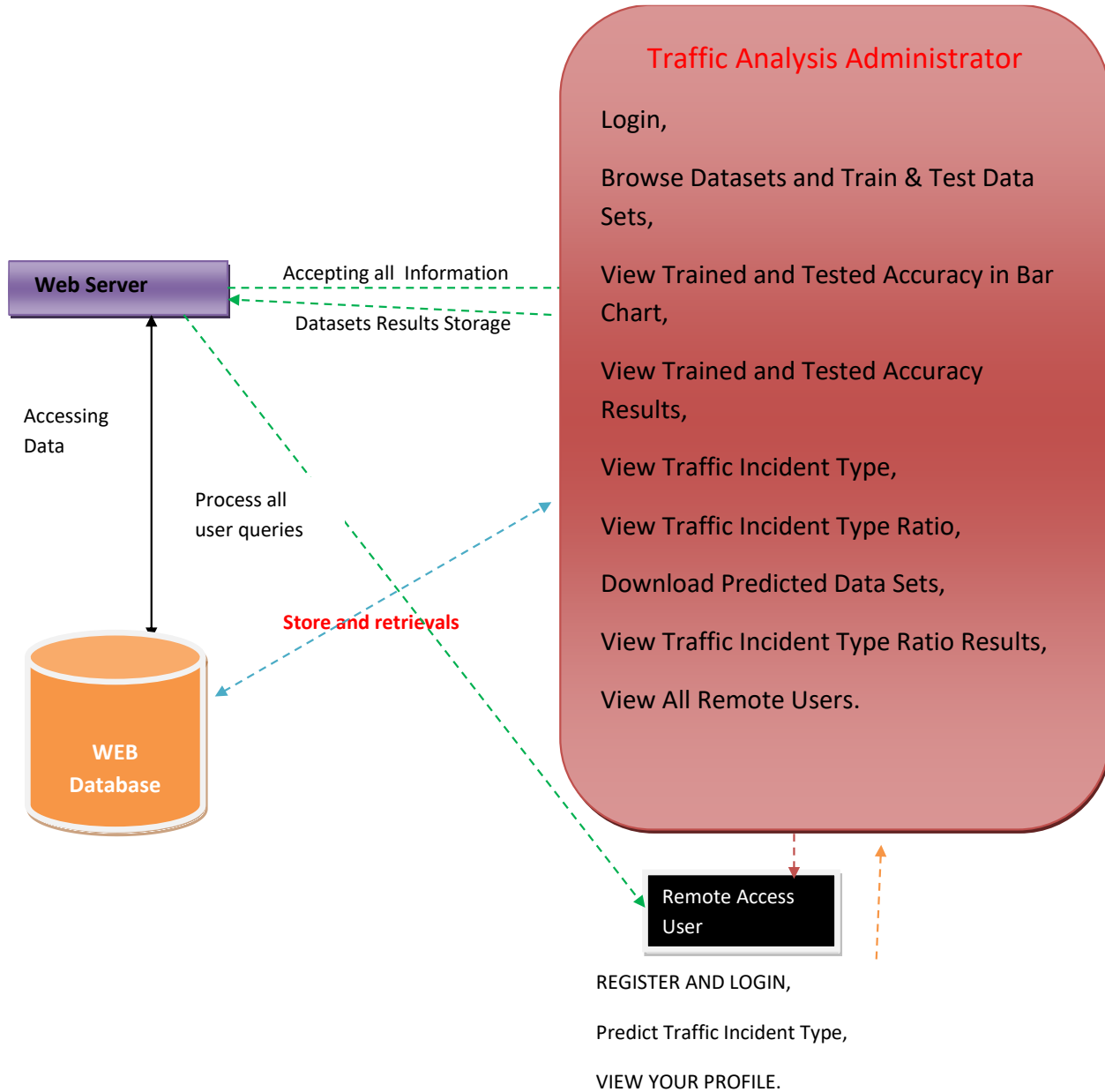
SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE



6.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

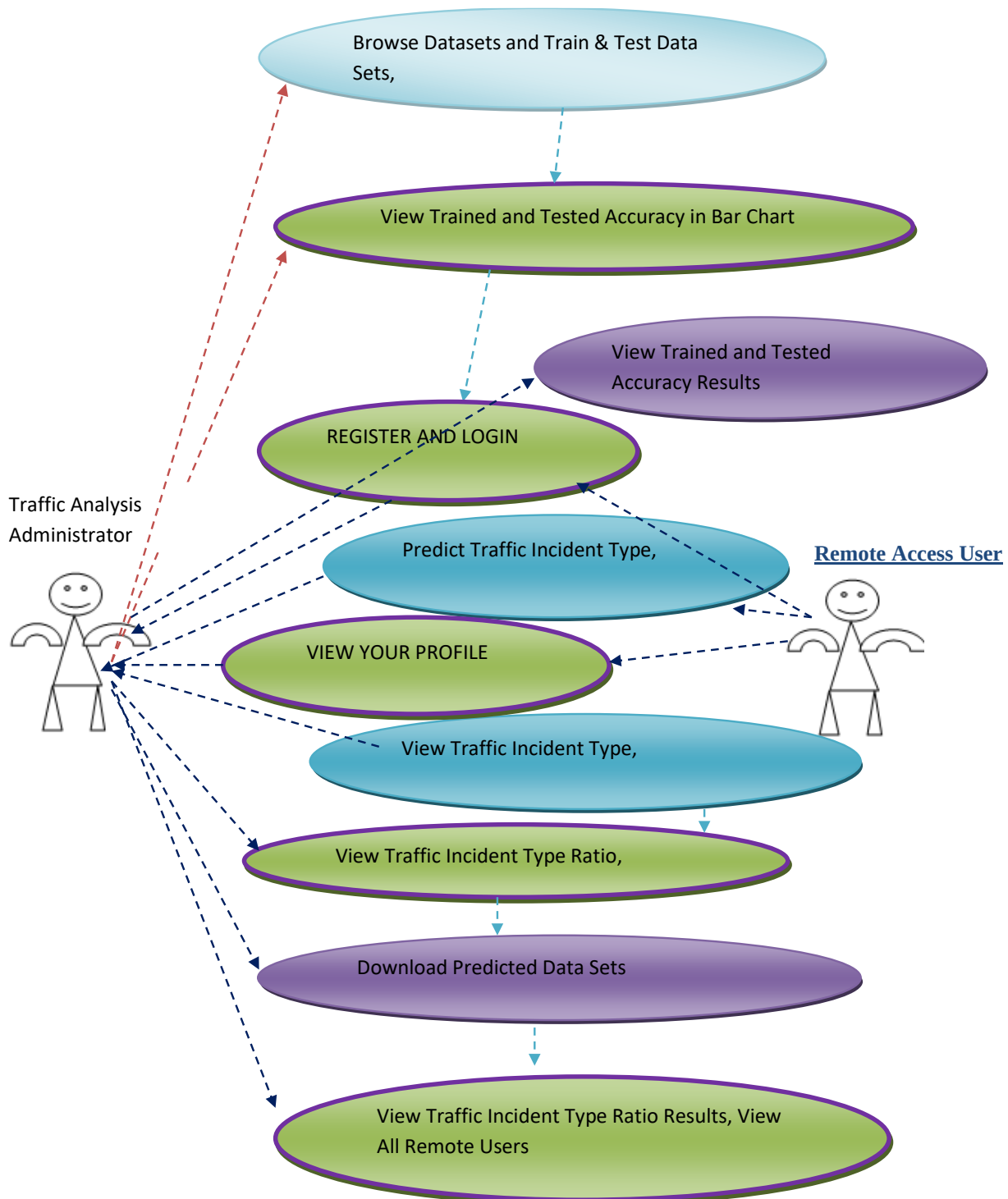
The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS

The Primary goals in the design of the UML are as follows:

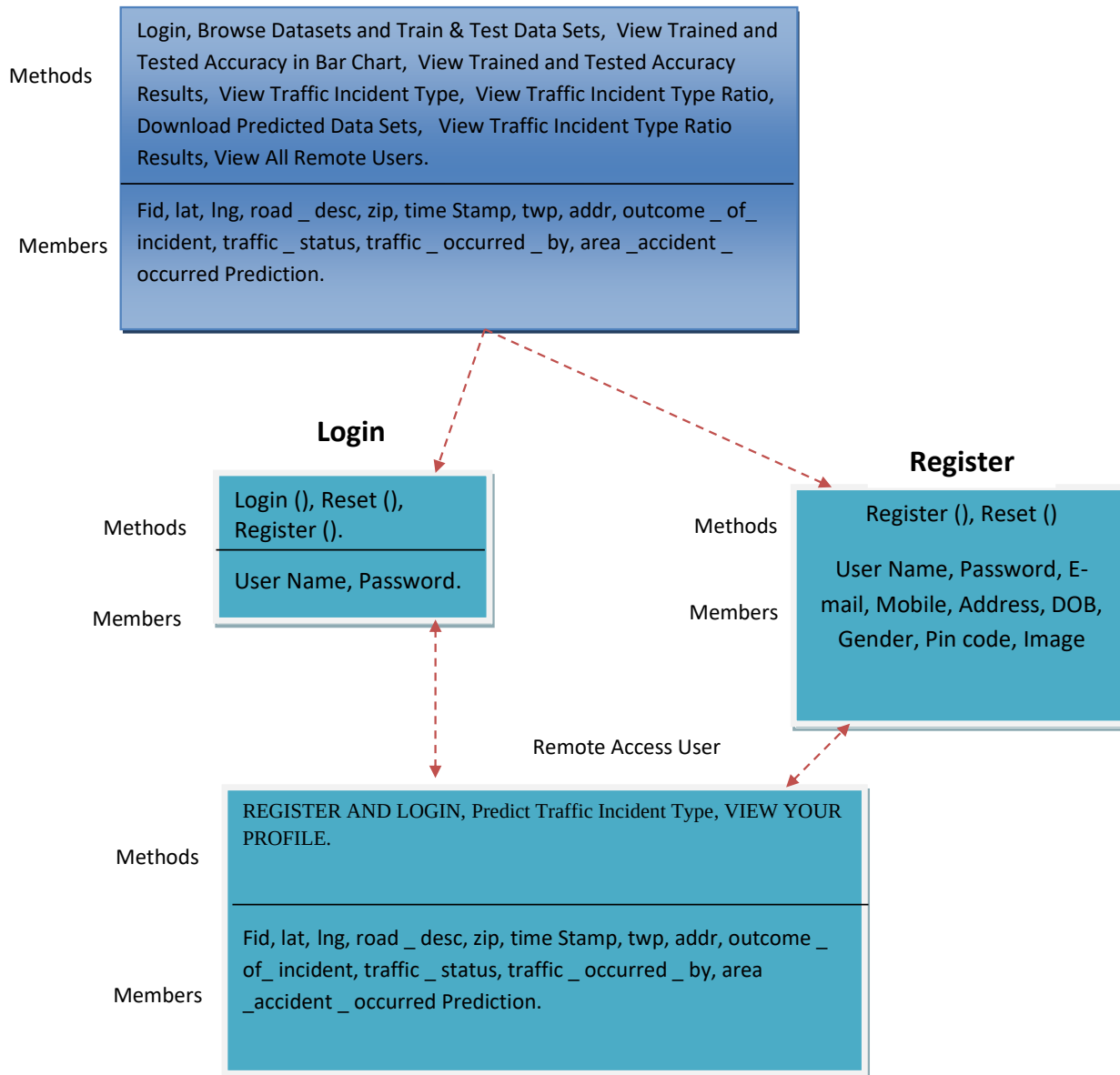
1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

6.2.1 USECASE DIAGRAM

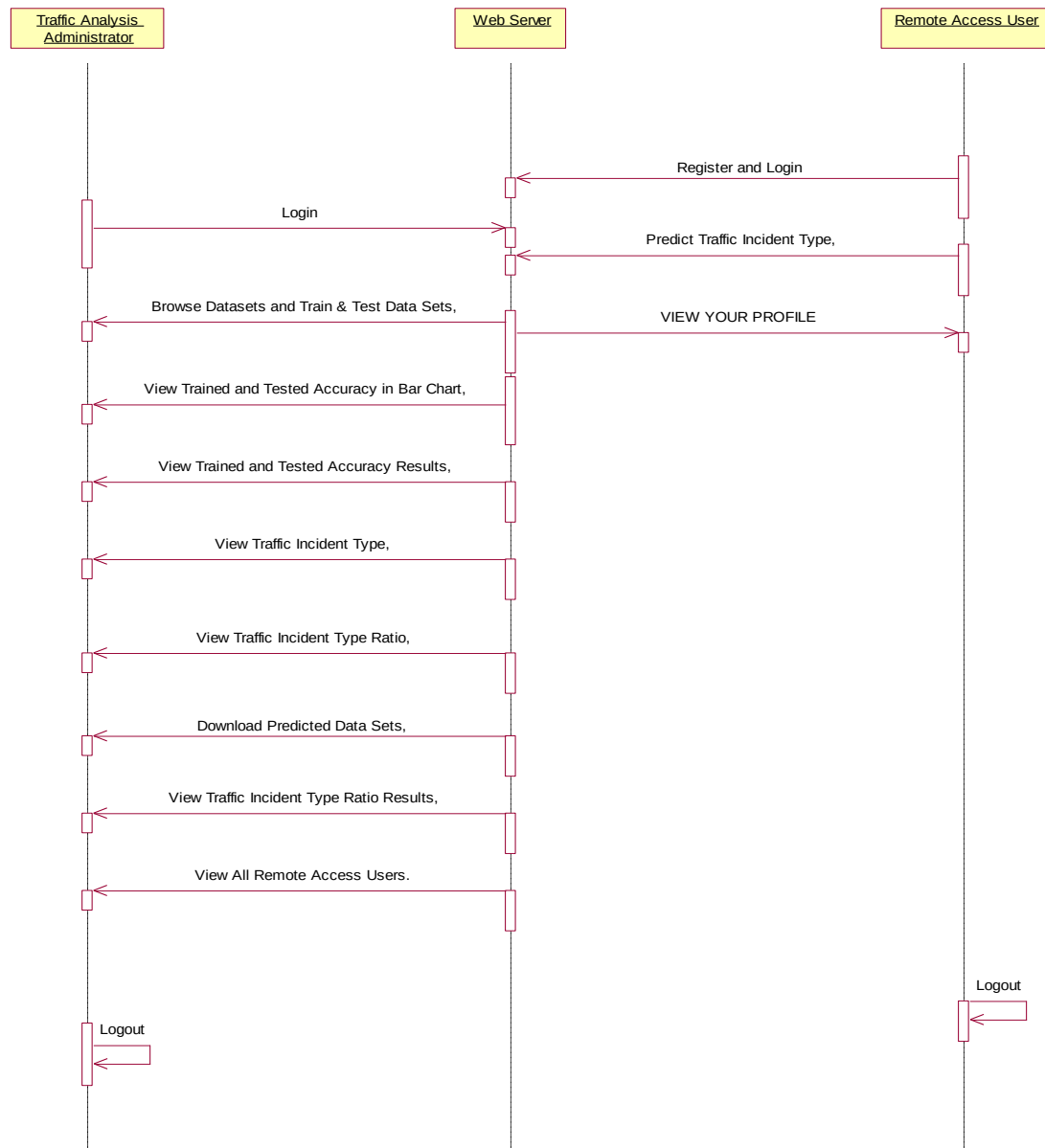


6.2.2 CLASS DIAGRAM

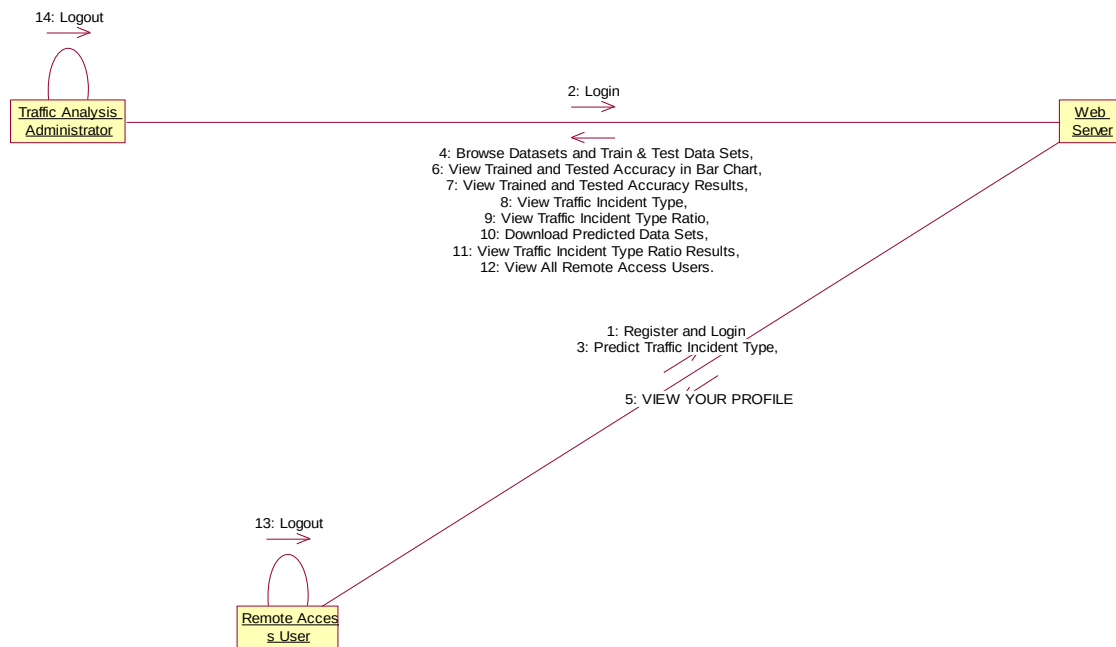
Traffic Analysis Administrator



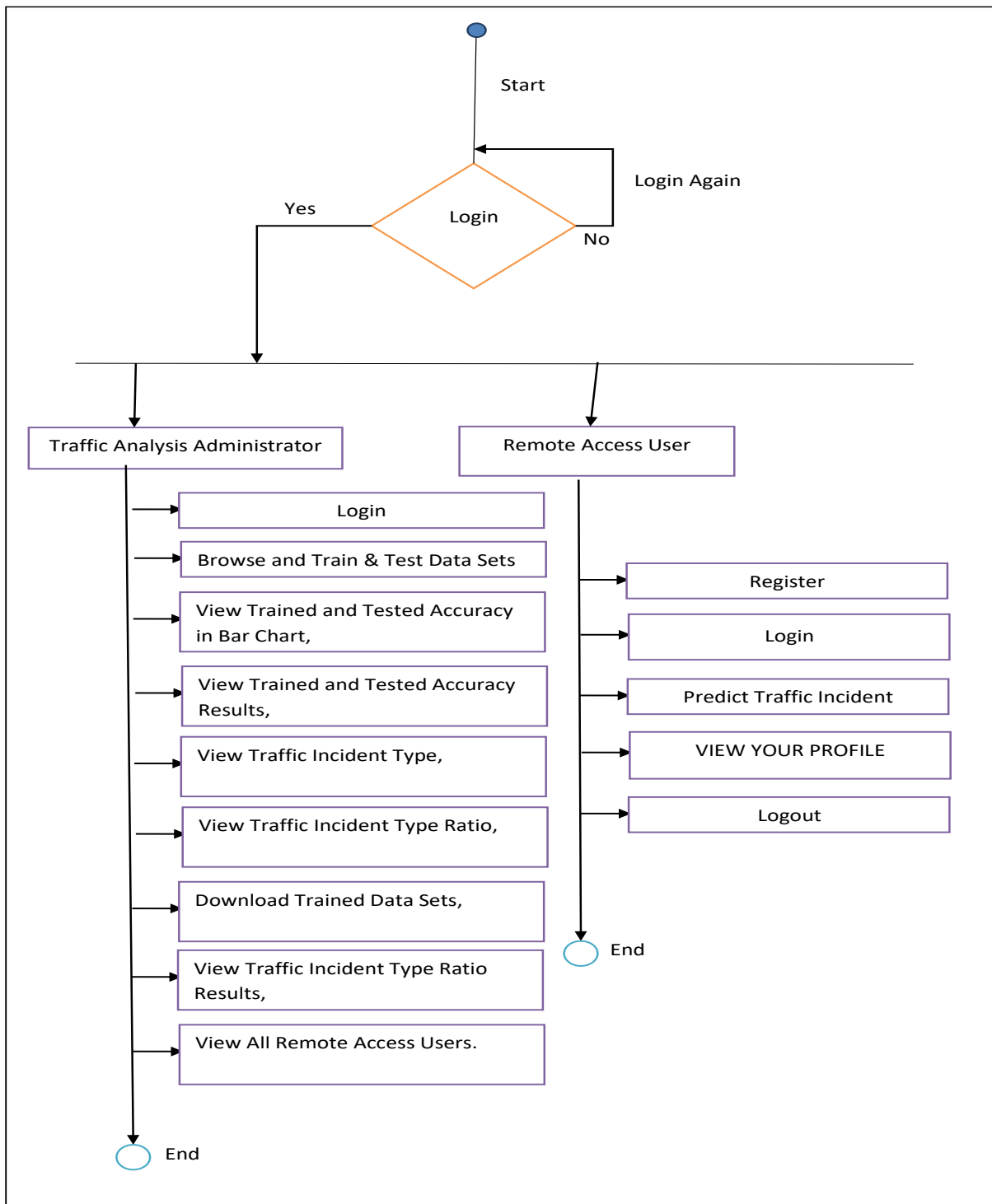
6.2.3 SEQUENCE DIAGRAM



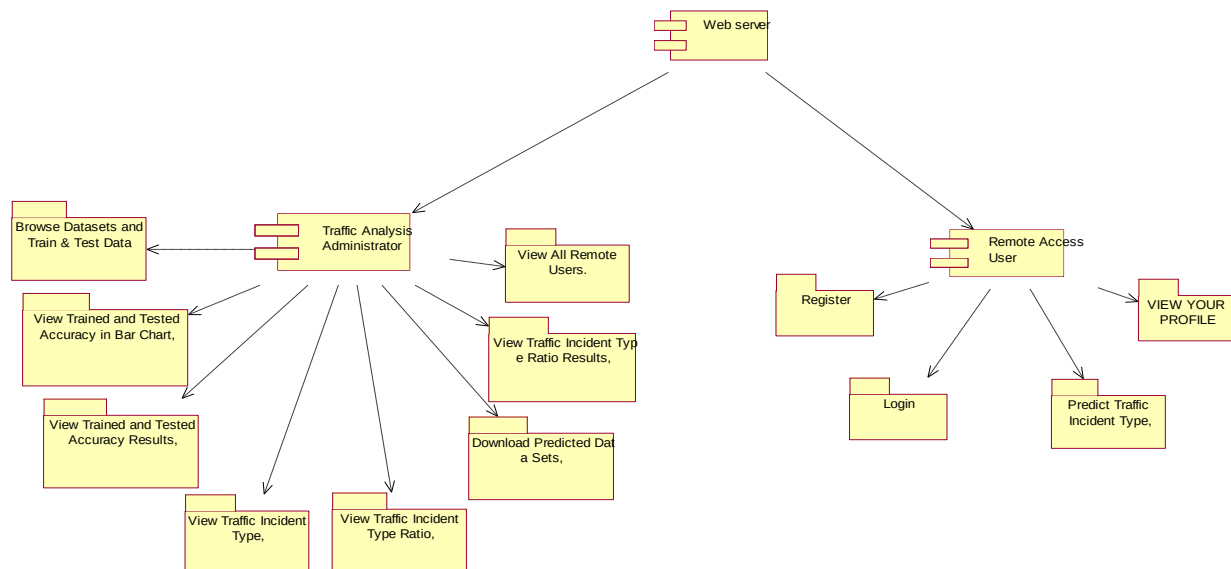
6.2.4 COLLABRATION DIAGRAM



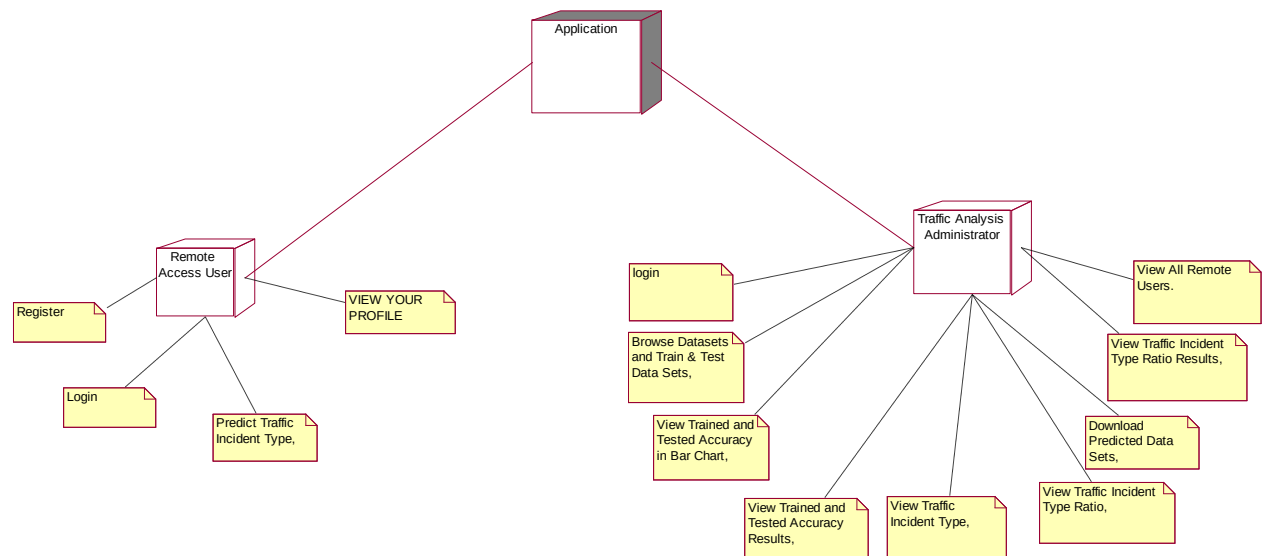
6.2.5 ACTIVITY DIAGRAM



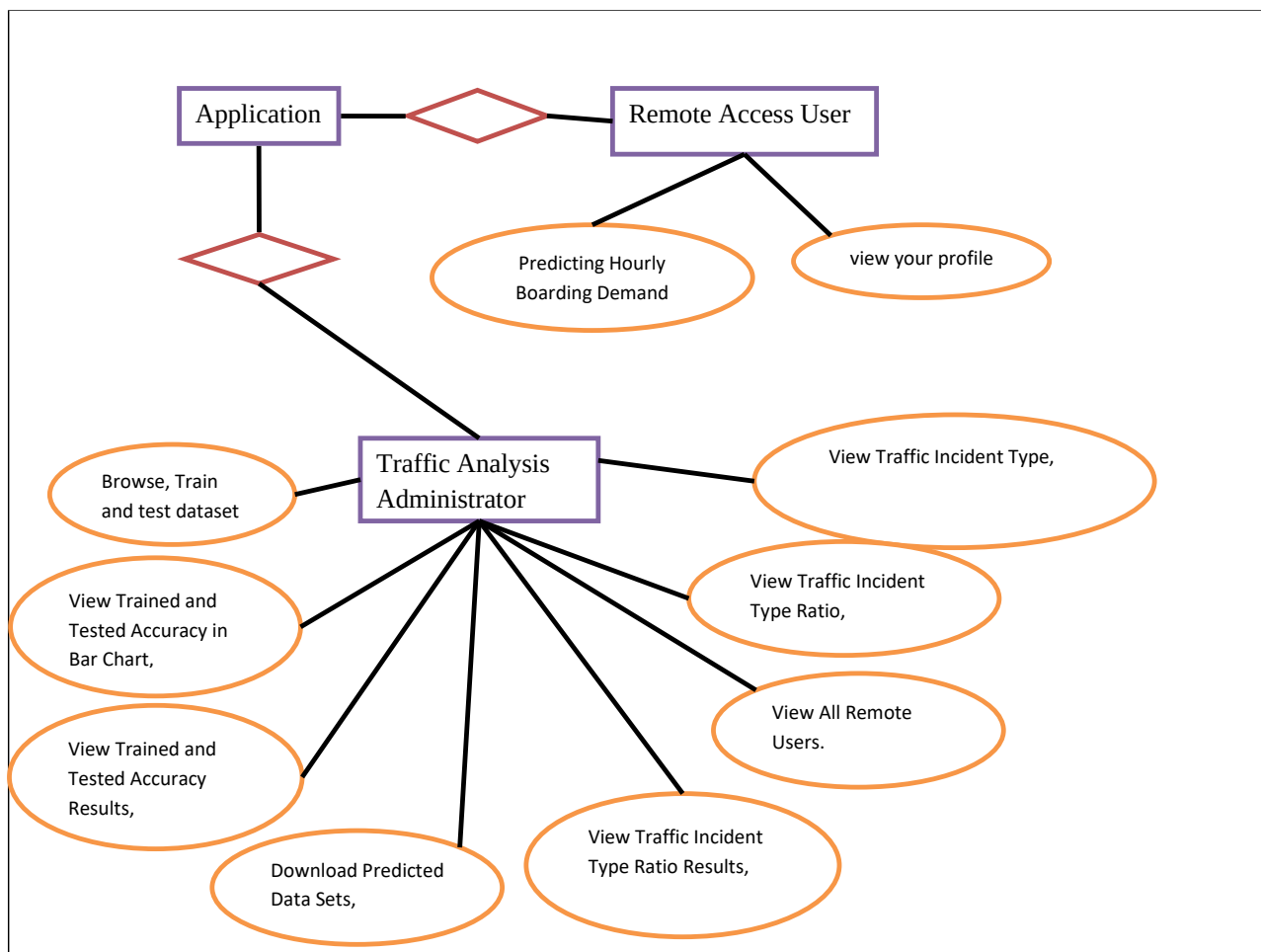
6..2.6 COMPONENT DIAGRAM



6.2.7 DEPLOYMENT DIAGRAM



6.2.8 ER DIAGRAM



6.2.9 DATA DICTIONARY

Database: datafits

Table name: auth_group

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
name	Varchar(1000)	Not null	name

Table Name: Auth_group_Permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Group id	Int(11)	Primary Key	Unique Identifier
Permission_id	Int(11)	Primary Key	Unique Identifier

Table Name: auth_permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Name	Int(255)	Not Null	name
Content_type_id	Int(11)	Primary Key	Unique Identifier
Code name	Int(100)	Not Null	name

Table Name: auth_User

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Password	varchar(128)	Not Null	password
Last_Login	Datetime(6)	Not Null	Last login
Is_superuser	tinyint(1)	Not Null	Name
username	Varchar(150)	Not Null	Username
lastname	Varchar(30)	Not Null	Lastname
email	Varchar(150)	Not Null	Email id
Is_staff	Tinyint(1)	Not Null	Staff
Is_active	Tinyint(1)	Not Null	Active
Date_joined	Datetime(6)	Not Null	Date and time

Table Name: auth_user_groups

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
User_id	Int(11)	Primary Key	Unique Identifier
Group_id	Int(11)	Primary Key	Unique Identifier

CHAPTER-7

INPUT AND OUTPUT DESIGN

7.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.1.1 OBJECTIVES

1.Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3.When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

7.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER-8

IMPLEMENTATION

8.1 MODULES

- Traffic Analysis Administrator
- Remote Access User

8.1.1 MODULE DESCRIPTION

Traffic Analysis Administrator

In this module, the Traffic Analysis Administrator has to login by using valid user name and password. After login successful he can do some operations such as Train & Test Agriculture Crop Data Sets, View Trained and Tested Accuracy in Bar Chart, View Trained and Tested Accuracy Results, View Prediction Of Agricultural Text Classification Type, View Agricultural Text Classification Type Ratio, Download Predicted Data Sets, View Agricultural Text Classification Type Ratio Results, View All Remote Users.

View and Authorize Users

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorizes the users.

REMOTE ACCESS USER

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like REGISTER AND LOGIN, Predict Agricultural _ text _ classification, VIEW YOUR PROFILE.

CHAPTER-9

SOFTWARE ENVIRONMENT

9.1 PYTHON

Python is a **high-level, interpreted, interactive** and **object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

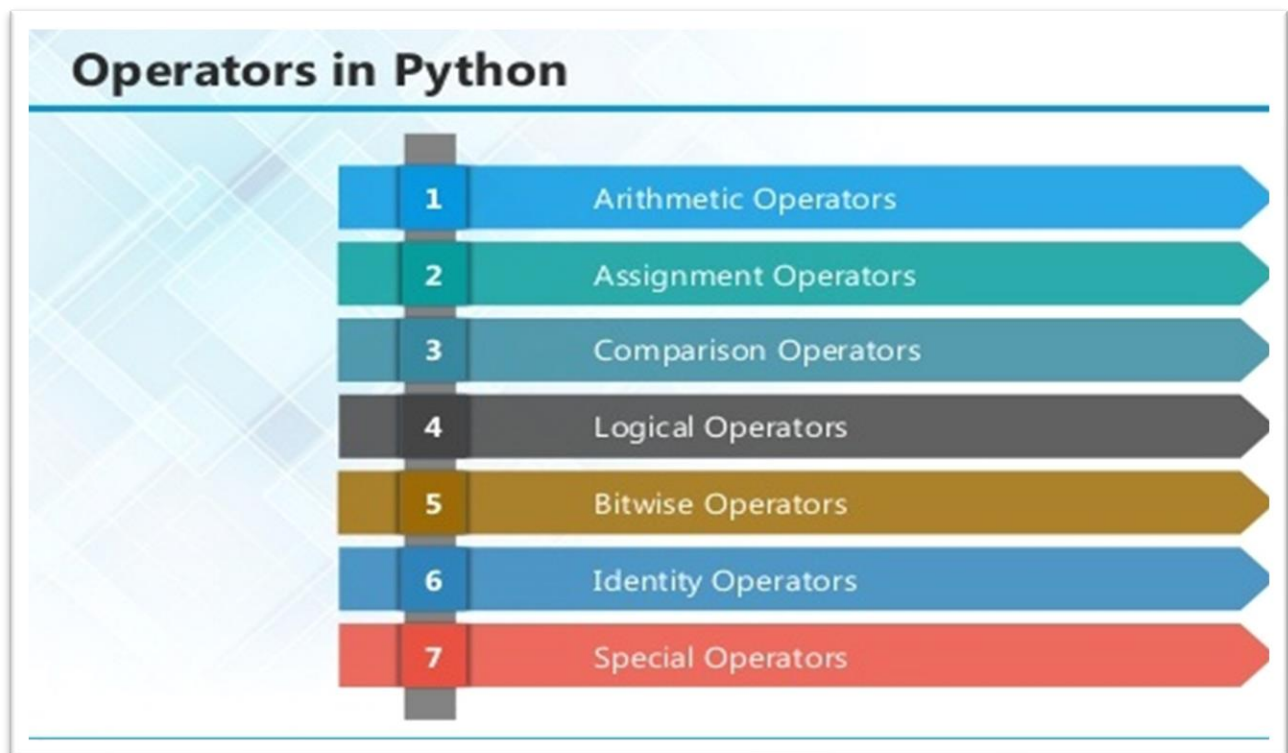
Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.

- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$, - $11 // 3 = -4$, - $11.0 // 3 = -4.0$

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$ $c /= a$ is equivalent to $c = c / a$

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	c %= a is equivalent to c = c % a
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	c **= a is equivalent to c = c ** a
//= Floor Division	It performs floor division on operators and assign value to the left operand	c //= a is equivalent to c = c // a

IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y)

COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];  
list2 = [1, 2, 3, 4, 5];  
list3 = ["a", "b", "c", "d"]
```

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
1	<u><code>cmp(list1, list2)</code></u> Compares elements of both lists.
2	<u><code>len(list)</code></u> Gives the total length of the list.
3	<u><code>max(list)</code></u> Returns item from the list with max value.
4	<u><code>min(list)</code></u> Returns item from the list with min value.
5	<u><code>list(seq)</code></u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	<u><code>list.append(obj)</code></u> Appends object obj to list
2	<u><code>list.count(obj)</code></u> Returns count of how many times obj occurs in list
3	<u><code>list.extend(seq)</code></u> Appends the contents of seq to list
4	<u><code>list.index(obj)</code></u> Returns the lowest index in list that obj appears
5	<u><code>list.insert(index, obj)</code></u> Inserts object obj into list at offset index
6	<u><code>list.pop(obj=list[-1])</code></u> Removes and returns last object or obj from list
7	<u><code>list.remove(obj)</code></u> Removes object obj from list
8	<u><code>list.reverse()</code></u> Reverses objects of list in place
9	<u><code>list.sort([func])</code></u> Sorts objects of list, use compare function if given

3.2 TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5);  
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

- Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5, 6, 7);  
print "tup1[0]: ", tup1[0]  
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);  
tup2 = (' abc ', 'xyz ');  
tup3 = tup1 + tup2;  
print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);  
print tup  
del tup;  
print "After deleting tup : "  
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description

<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	<code>(1, 2, 3, 4, 5, 6)</code>	Concatenation
<code>('Hi!') * 4</code>	<code>('Hi!', 'Hi!', 'Hi!', 'Hi!')</code>	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1, 2, 3): print x,</code>	1 2 3	Iteration

Built-in Tuple Functions

SN	Function with Description
1	cmp(tuple1, tuple2): Compares elements of both tuples.
2	len(tuple): Gives the total length of the tuple.
3	max(tuple): Returns item from the tuple with max value.
4	min(tuple): Returns item from the tuple with min value.
5	tuple(seq): Converts a list into tuple.

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara
dict['Age']: 7
```

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

del dict['Name']; # remove entry with key 'Name'

dict.clear();    # remove all entries in dict

del dict;        # delete entire dictionary

print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods—

Python includes the following dictionary functions —

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	dict.clear(): Removes all elements of dictionary <i>dict</i>
2	dict. Copy(): Returns a shallow copy of dictionary <i>dict</i>
3	dict.fromkeys(): Create a new dictionary with keys from seq and values set to <i>value</i> .
4	dict.get(key, default=None): For <i>key</i> key, returns value or default if key not in dictionary
5	dict.has_key(key): Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	dict.items(): Returns a list of <i>dict</i> 's (key, value) tuple pairs
7	dict.keys(): Returns list of dictionary dict's keys

8	dict.setdefault(key, default=None): Similar to <code>get()</code> , but will set <code>dict[key]=default</code> if <code>key</code> is not already in <code>dict</code>
9	dict.update(dict2): Adds dictionary <code>dict2</code> 's key-values pairs to <code>dict</code>
10	dict.values(): Returns list of dictionary <code>dict</code> 's values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like `print()`, etc. but you can also create your own functions. These functions are called *user-defined functions*.

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses `(( ))`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon `(:)` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

```
def functionname( parameters ):
    "function_docstring"
```

```
function_suite  
  
return [expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function –

```
# Function definition is here  
  
def printme( str ):      
    "This prints a passed string into this function"  
    print str  
    return;  
  
# Now you can call printme function  
  
printme("I'm first call to user defined function!")  
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!  
Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example –

—

```

total = 0; # This is global variable.

# Function definition is here
def sum( arg1, arg2 ):
    # Add both the parameters and return them."
    total = arg1 + arg2; # Here total is local variable.
    print "Inside the function local total : ",total
    return total;
sum( 10, 20 );
print "Outside the function global total : ",total

```

Result –

```

Inside the function local total : 30
Outside the function global total : 0

```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```

def print_func( par ):
    print "Hello : ", par
    return

```

The *import* Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module `support.py`, you need to put the following command at the top of the script –

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code –

```
def Pots():  
    print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above –

- *Phone/Isdn.py* file having function `Isdn()`
- *Phone/G3.py* file having function `G3()`

Now, create one more file `__init__.py` in *Phone* directory –

- *Phone/__init__.py*

To make all of your functions available when you've imported Phone, to put explicit import statements in `__init__.py` as follows –

```
from Pots import Pots
from Isdn import Isdn
from G3 import G3
```

After you add these lines to `__init__.py`, you have all of these classes available when you import the Phone package.

```
# Now import your Phone Package.
import Phone
Phone.Pots()
Phone.Isdn()
Phone.G3()
```

RESULT:

```
I'm Pots Phone
I'm 3G Phone
I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and

writes the result to standard output as follows –

```
print "Python is really a great language,","isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- `raw_input`
- `input`

The `raw_input` Function

The `raw_input([prompt])` function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");  
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this –

```
Enter your input: Hello Python  
Received input is : Hello Python
```

The `input` Function

The `input([prompt])` function is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: ");  
print "Received input is : ", str
```

This would produce the following result against the entered input –

```
Enter your input: [x*5 for x in range(2,10,2)]
```

```
Recieved input is : [10, 20, 30, 40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.

ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The *file* Object Attributes

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
```



```
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
print "Softspace flag : ", fo.softspace
```

This produces the following result –

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
Softspace flag : 0
```

The `close()` Method

The `close()` method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done. Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the `close()` method to close a file

Syntax

```
fileObject.close();
```

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
# Close opened file
fo.close()
```

Result –

```
Name of the file: foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use `read()` and `write()` methods to read and write files.

The `write()` Method

The `write()` method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The `write()` method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. **Example**

```
# Open a file
fo = open("foo.txt", "wb")
fo.write("Python is a great language.\nYeah its great!!\n");

# Close opened file
fo.close()
```

The above method would create `foo.txt` file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.
Yeah its great!!
```

The `read()` Method

The `read()` method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if `count` is missing, then it tries to read as much as possible, maybe until the end of file.

Example

Let's take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
print "Read String is : ", str
# Close opened file
fo.close()
```

This produces the following result –

```
Read String is : Python is
```

File Positions

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

32

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example

Let us take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
```

```

print "Read String is : ", str

# Check current position
position = fo.tell();
print "Current file position : ", position

# Reposition pointer at the beginning once again
position = fo.seek(0, 0);
str = fo.read(10);
print "Again read String is : ", str

# Close opened file
fo.close()

```

This produces the following result –

```

Read String is : Python is
Current file position : 10
Again read String is : Python is

```

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method

The `rename()` method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example

Following is the example to rename an existing file *test1.txt*:

```
import os

# Rename a file from test1.txt to test2.txt
os.rename( "test1.txt", "test2.txt" )
```

The *remove()* Method

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Example

Following is the example to delete an existing file *test2.txt* –

```
#!/usr/bin/python
import os

# Delete file test2.txt
os.remove("test2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The **os** module has several methods that help you create, remove, and change directories.

The *mkdir()* Method

You can use the *mkdir()* method of the **os** module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory *test* in the current directory –

```
#!/usr/bin/python
import os

# Create a directory "test"
os.mkdir("test")
```

The *chdir()* Method

You can use the *chdir()* method to change the current directory. The *chdir()* method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into *"/home/newdir"* directory –

```
#!/usr/bin/python
import os

# Changing a directory to "/home/newdir"
os.chdir("/home/newdir")
```

The *getcwd()* Method

The *getcwd()* method displays the current working directory.

Syntax

```
os.getcwd()
```

Example

Following is the example to give current directory –

```
import os

# This would give location of the current directory

os.getcwd()
```

The *rmdir()* Method

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example

Following is the example to remove `"/tmp/test"` directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os

# This would remove "/tmp/test" directory.

os.rmdir( "/tmp/test" )
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling:** This would be covered in this tutorial. Here is a list standard Exceptions available in Python: [Standard Exceptions](#).
- **Assertions:** This would be covered in [Assertions in Python](#)

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.

AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.

IOError IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist. Raised for operating system-related errors.
SyntaxError IndentationError	Raised when there is an error in Python syntax. Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle
- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection
-

9.2 SOURCE CODE

MANAGE.PY

```
#!/usr/bin/env python
```

```
"""Django's command-line utility for administrative tasks."""
```

```
import os
```

```
import sys
```

```
def main():
```

```
    """Run administrative tasks."""
```

```
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'datafits.settings')
```

```
    try:
```

```
        from django.core.management import execute_from_command_line
```

```
    except ImportError as exc:
```

```
        raise ImportError(
```

```

    "Couldn't import Django. Are you sure it's installed and "
    "available on your PYTHONPATH environment variable? Did you "
    "forget to activate a virtual environment?"

) from exc

execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

```

SERVICE PROVIDER LOGIN PAGE

```

<link href="//maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css" rel="stylesheet"
id="bootstrap-css">

{% load static %}

<!DOCTYPE html>

<html lang="en">

    <title>Login</title>

    <meta charset="utf-8">

    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">

    <head>

<link rel="icon" href="images/icon.png" type="image/x-icon" />

```

```
<link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">

<link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">

<link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">
```

```
<style>

    body {background: url("{% static 'bg.jpg' %}");

    background-color:#eee;}

    .container-fluid {padding:50px;}

    .container{background-color:white;padding:50px; }

    #title{font-family: 'Fredoka One', cursive;

}

    .text-uppercase{

    font-family: 'Righteous', cursive;

    }

    .style1 {

    color: #FF0000;

    font-weight: bold;

}

    .style2 {

    font-family: 'Righteous', cursive;

    color: #FF0000;
```

```

        font-weight: bold;
    }

    .style5 {
        color: #FF0099;
        font-weight: bold;
    }

</style>

</head>

<body>

<div class="container-fluid">

    <div>

        <h2 class="text-center style1" id="title">DataFITS A Heterogeneous Data
Fusion Framework for Traffic and Incident Prediction</h2>

        <p      class="text-center">      <span      class="style5"><small
id="passwordHelpInline" class="text-muted">Intelligent transportation systems, heterogeneous data
fusion, traffic estimation, incident classification.

systems.

        </small></span> </p>

        <hr>

        <div class="row">

            <div class="col-md-2">

```

```

<!--null-->

</div>

<div class="col-md-5">

    <form method="POST" role="form">

        {% csrf_token %}

        <fieldset>

            {% load static %}

            <p class="style2"> Login Service Provider:

</p>

            <div class="form-group">

                <input type="text" name="username"

placeholder="User Name" required>

            </div>

            <div class="form-group">

                <input type="password"

name="password" placeholder="Password" required>

            </div>

            <div>

                <input type="submit" name="submit1"

class="btn btn-md" value="Login">

            </div>

        </fieldset>

```



```

<br><br>
<button class="btn btn-lg "><a href="{% url 'login'
%}">User Login</a></button>

</form>
</div>

</div>

</div>

</div>

</body>

</html>

```

CHAPTER-10

RESULTS/DISCUSSION

10.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

WHITE BOX TESTING

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

10.1.1 TEST CASES:

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for User Registration form:

FUNCTION	REMOTE USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.

LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password:

When the old password does not match with the new password , then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
User	Login Usecase	1. Navigate To Www.Sample.Co m 2. Click On Submit Button Without Entering Username and Password	A Validation Should Be As Below “Please Enter Valid Username & Password”	A Validation Has Been Populated As Expected	Pass	High

		1. aNavigate To Www.Sample.Co m 2. Click On Submit Button With Out Filling Password And With Valid Username	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “	A Validation Is Shown As Expected	Pass	High
		1. NNavigate To Www.Sample.Co m 2. Enter Both Username And Password Wrong And Hit Enter	A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected	Pass	High
		1. Navigate To Www.Sample.Co m 2. Enter Validate Username And Password And Click On Submit	Validate Username And Password In DataBase And Once If They Correct Then Show The Main Page	Main Page/ Home Page Has Been Displayed	Pass	High

10.2 SCREENSHOTS

```
ault 'django.db.models.AutoField'.
  HINT: Configure the DEFAULT_AUTO_FIELD setting or the ClientSiteConfig.default_auto_field attribute to point to
a subclass of AutoField, e.g. 'django.db.models.BigAutoField'.
Remote_User.detection_ratio: (models.W042) Auto-created primary key used when not defining a primary key type, by default
t 'django.db.models.AutoField'.
  HINT: Configure the DEFAULT_AUTO_FIELD setting or the ClientSiteConfig.default_auto_field attribute to point to
a subclass of AutoField, e.g. 'django.db.models.BigAutoField'.
Remote_User.traffic_incident_prediction: (models.W042) Auto-created primary key used when not defining a primary key typ
e, by default 'django.db.models.AutoField'.
  HINT: Configure the DEFAULT_AUTO_FIELD setting or the ClientSiteConfig.default_auto_field attribute to point to
a subclass of AutoField, e.g. 'django.db.models.BigAutoField'.

System check identified 4 issues (0 silenced).

You have 4 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth.
Run 'python manage.py migrate' to apply them.
May 18, 2024 - 14:30:54
Django version 3.2.25, using settings 'datafits.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
[18/May/2024 14:30:58] "GET / HTTP/1.1" 200 5377
[18/May/2024 14:30:58] "GET /static/coin-slider.css HTTP/1.1" 404 1804
[18/May/2024 14:30:58] "GET /static/script.js HTTP/1.1" 404 1786
[18/May/2024 14:30:58] "GET /static/cufon-aller.js HTTP/1.1" 404 1801
[18/May/2024 14:30:58] "GET /static/style.css HTTP/1.1" 404 1786
[18/May/2024 14:30:58] "GET /static/cufon-yui.js HTTP/1.1" 404 1795
[18/May/2024 14:30:58] "GET /static/jquery-1.4.2.min.js HTTP/1.1" 404 1816
[18/May/2024 14:30:58] "GET /static/coin-slider.min.js HTTP/1.1" 404 1813
```

Fig 1: Command Prompt

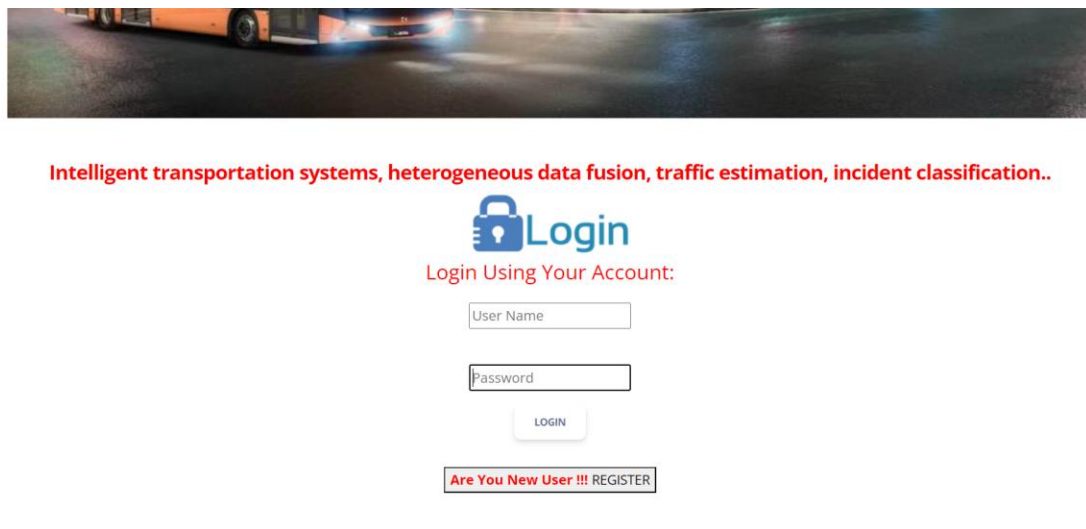


Fig 3: Remote Access User Login Page

Intelligent transportation systems, heterogeneous data fusion, traffic estimation, incident classification. systems.

REGISTER NOW!

REGISTER YOUR DETAILS HERE !!!

Enter Username	User Name	Enter Password	Password
Enter EMAIL Id	Enter Email	Enter Address	Enter Address
Enter Gender	-----Select Gender -----	Enter Mobile Number	Enter Mobile Number
Enter Country Name	Enter Country Name	Enter State Name	Enter State Name
Enter City Name	Enter City Name	REGISTER	

Registered Status ::

Fig 4: Remote Access User Registration Page



Fig 5 : Traffic Analysis Administrator Login Page

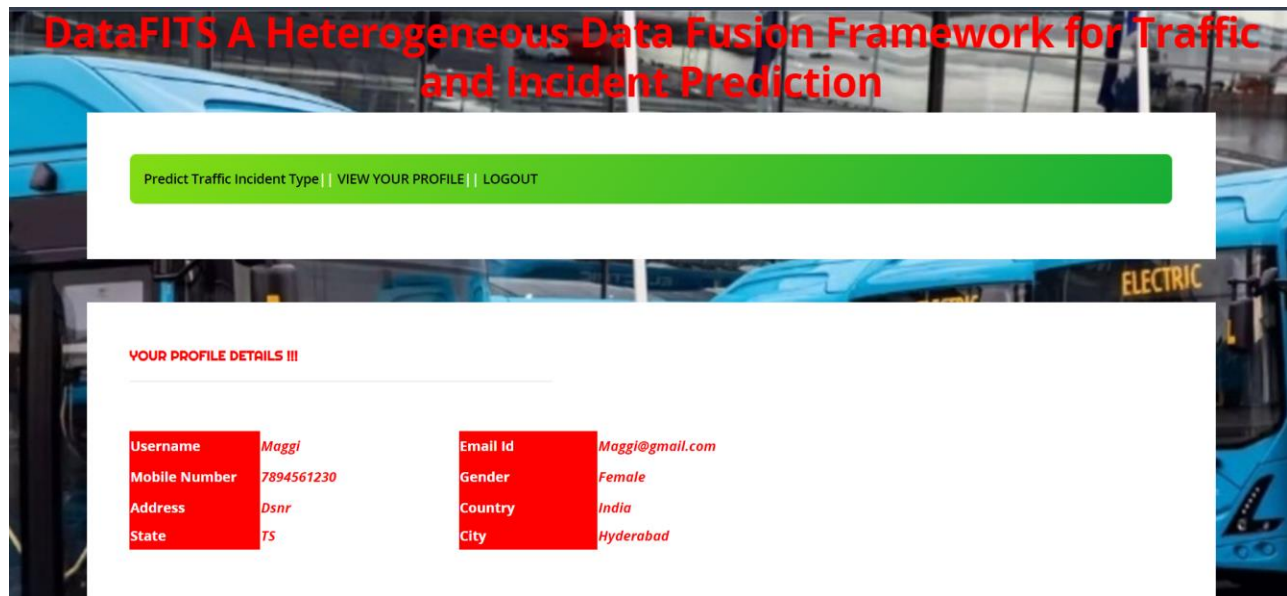


Fig 6 : Remote Access User Home Page



Fig 7 : Remote Access User Predict of Traffic Incident Type operation page

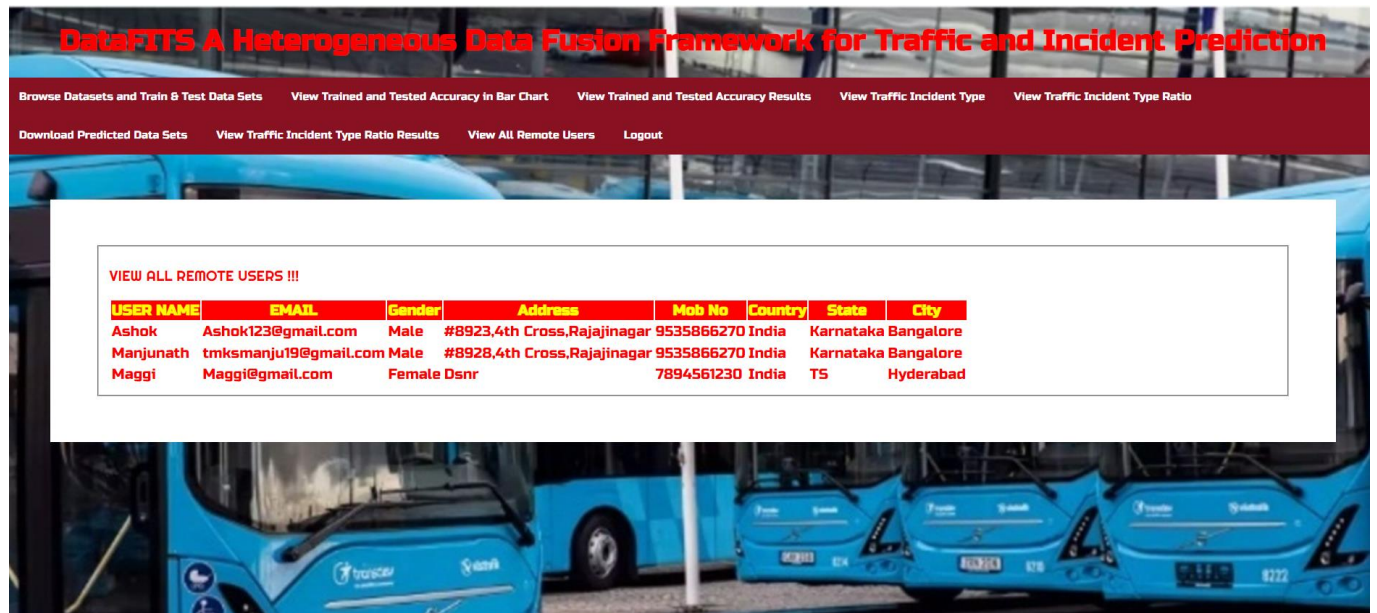


Fig 8 : View All Remote Access Users Page

CHAPTER-11

CONCLUSION

11.1 CONCLUSION

In this paper, we introduce Data FITS, an open-source data fusion framework that integrates diverse data by collecting, analyzing, and fusing it. We hypothesize that heterogeneous data fusion increases data quantity and quality, thereby improving datasets for ITS applications. To verify this, we developed two ITS applications: one used polynomial regression to estimate traffic levels, while the other combined traffic and incident data to classify events into accident, congestion, or non-incidents. Using real heterogeneous data from two German cities, we quantified the advantages of Data FITS by compiling a fused dataset. Our results indicate that Data FITS integrated data from multiple sources for 40% of all roads, thereby increasing the overall road coverage by 137%. In addition, the traffic estimation model, which uses polynomial regression, outperformed our previous approach based on descriptive statistics, achieving a high R^2 score of 0.91, low error metrics of 0.05, and provides accurate traffic estimations using the fused dataset. Compared to using a single sources dataset, the fused dataset estimation showed minor accuracy improvements but drastically improved the spatiotemporal coverage of the estimated areas. Our incident classification model relies on the fusion of traffic and incident data, achieving a 90% binary classification accuracy rate within our evaluation. Preprocessing the data, such as removing unclear traffic patterns, improved accuracy by an average of 29% . The classification of incidents into different categories resulted in a slightly lower accuracy of 86%, with unequal performance among classes indicated by F1 scores. To mitigate this problem, we oversampled the training dataset to create a more uniform representation of the data, resulting in an 80% accuracy for each class. Collecting more accident data can also solve this problem. We plan to expand the Data FITS framework by collecting and fusing more data types, improving its performance and data quality, and expanding its data analysis. We focus on data types such as social media and images, which require methods such as Natural Language Processing (NLP) and image processing. For ITS applications, we aim to use automated machine learning to explore different models and hyper-parameters and compare them with our current models. We also plan to analyze the correlation between traffic and incidents and incorporate it into the traffic estimation models. In addition, we intend to explore the use of big data in military scenarios, combining information from the civilian and military fields to support strategic operations in urban warfare. To this end, our framework can be enhanced to collect and combine different types of information

(image, text) to create common operational pictures and verify/authenticate information, thereby avoiding misinformation that may influence political decisions.

11.2 Future Scope

While DataFITS represents a significant advancement in traffic and incident prediction, there are several avenues for future research and development to further enhance its capabilities and applicability:

- **Integration of Emerging Technologies:** Future iterations of DataFITS could explore the integration of emerging technologies such as edge computing, Internet of Things (IoT) sensors, and connected vehicle data to improve data collection, processing speed, and prediction accuracy.
- **Enhancement of Machine Learning Models:** Continual refinement and optimization of machine learning models within DataFITS can lead to improved prediction accuracy, scalability, and robustness in handling diverse data sources and evolving traffic patterns.
- **Incorporation of Real-Time Feedback:** Integrating mechanisms for real-time feedback and adaptation based on observed outcomes can enhance the agility and responsiveness of DataFITS, enabling it to dynamically adjust predictions and recommendations in response to changing conditions.
- **Extension to Multi-Modal Transportation:** Expanding the scope of DataFITS to encompass multi-modal transportation systems, including public transit, cycling, and pedestrian infrastructure, can provide a more comprehensive view of urban mobility and facilitate integrated transportation planning and management.

CHAPTER-12

REFERENCES

- [1] L. Zhu, F. R. Yu, Y. Wang, B. Ning, and T. Tang, “Big data analytics in intelligent transportation systems: A survey,” *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 1, pp. 383–398, Jan. 2019.
- [2] Umweltbundesamt. (2022). Verkehrsinfrastruktur und fahrzeugbestand. Accessed: Dec. 12, 2022. [Online]. Available: <https://www.umweltbundesamt.de/daten/verkehr/verkehrsinfrastrukturfahrzeugbestand>
- [3] German Federal Statistical Office (Destatis). (2022). Passengers Carried in Germany. Accessed: Jul. 12, 2022. [Online]. Available: <https://www.destatis.de/EN/Themes/Economic-Sectors-Enterprises/Transport/Passenger-Transport/Tables/passengerscarried.html>
- [4] G. Vitor, P. Rito, and S. Sargento, “Smart city data platform for real-time processing and data sharing,” in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Sep. 2021, pp. 1–7.
- [5] A. B. Campolina, P. H. L. Rettore, M. Do Val Machado, and A. A. F. Loureiro, “On the design of vehicular virtual sensors,” in *Proc. 13th Int. Conf. Distrib. Comput. Sensor Syst. (DCOSS)*, Jun. 2017, pp. 134–141.
- [6] S. Jeong, S. Kim, and J. Kim, “City data hub: Implementation of standard-based smart city data platform for interoperability,” *Sensors*, vol. 20, no. 23, p. 7000, Dec. 2020. [Online]. Available: <https://www.mdpi.com/1424-8220/20/23/7000>
- [7] L. Zhang, Y. Xie, L. Xidao, and X. Zhang, “Multi-source heterogeneous data fusion,” in *Proc. Int. Conf. Artif. Intell. Big Data (ICAIBD)*, May 2018, pp. 47–51.
- [8] P. H. L. Rettore, B. P. Santos, A. B. Campolina, L. A. Villas, and A. A. F. Loureiro, “Towards intra-vehicular sensor data fusion,” in *Proc. IEEE 19th Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2016, pp. 126–131.
- [9] P. H. L. Rettore, A. B. Campolina, L. A. Villas, and A. A. F. Loureiro, “A method of eco-driving based on intra-vehicular sensor data,” in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2017, pp. 1122–1127.
- [10] P. H. L. Rettore, A. B. Campolina, A. Souza, G. Maia, L. A. Villas, and A. A. F. Loureiro, “Driver authentication in VANETs based on intravehicular sensor data,” in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2018, pp. 00078–00083.

- [11] G. L. Foresti, M. Farinosi, and M. Vernier, "Situational awareness in smart environments: Socio-mobile and sensor data fusion for emergency response to disasters," *J. Ambient Intell. Humanized Comput.*, vol. 6, no. 2, pp. 239–257, Apr. 2015.
- [12] H. Wen, Y. Lin, and J. Wu, "Co-evolutionary optimization algorithm based on the future traffic environment for emergency rescue path planning," *IEEE Access*, vol. 8, pp. 148125–148135, 2020.
- [13] P. H. Rettore, G. Maia, L. A. Villas, and A. A. F. Loureiro, "Vehicular data space: The data point of view," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2392–2418, 3rd Quart., 2019.
- [14] S. A. Kashinath et al., "Review of data fusion methods for realtime and multi-sensor traffic flow analysis," *IEEE Access*, vol. 9, pp. 51258–51276, 2021.
- [15] W. Jiang and J. Luo, "Big data for traffic estimation and prediction: A survey of data and tools," *Appl. Syst. Innov.*, vol. 5, no. 1, p. 23, Feb. 2022.
- [16] R. A. Anand, L. Vanajakshi, and S. C. Subramanian, "Traffic density estimation under heterogeneous traffic conditions using data fusion," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2011, pp. 31–36.
- [17] A. Abadi, T. Rajabioun, and P. A. Ioannou, "Traffic flow prediction for road transportation networks with limited traffic data," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 653–662, Apr. 2015.
- [18] G. Meena, D. Sharma, and M. Mahrishi, "Traffic prediction for intelligent transportation system using machine learning," in *Proc. 3rd Int. Conf. Emerg. Technol. Comput. Eng., Mach. Learn. Internet Things (ICETCE)*, Feb. 2020, pp. 145–148.
- [19] L. Zhao et al., "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3848–3858, Sep. 2020.
- [20] J. Tang, L. Li, Z. Hu, and F. Liu, "Short-term traffic flow prediction considering spatio-temporal correlation: A hybrid model combining type-2 fuzzy C-means and artificial neural network," *IEEE Access*, vol. 7, pp. 101009–101018, 2019.
- [21] X. Di, Y. Xiao, C. Zhu, Y. Deng, Q. Zhao, and W. Rao, "Traffic congestion prediction by spatiotemporal propagation patterns," in *Proc. 20th IEEE Int. Conf. Mobile Data Manage. (MDM)*, Jun. 2019, pp. 298–303.
- [22] X. Wang, X. Guan, J. Cao, N. Zhang, and H. Wu, "Forecast network-wide traffic states for multiple steps ahead: A deep learning approach considering dynamic non-local spatial correlation and non-stationary temporal dependency," *Transp. Res. C, Emerg. Technol.*, vol. 119, Oct. 2020, Art. no. 102763. [Online]. Available:

<https://www.sciencedirect.com/science/article/pii/S0968090X20306756>

- [23] C. Zheng, X. Fan, C. Wang, and J. Qi, “GMAN: A graph multi-attention network for traffic prediction,” in Proc. AAAI, vol. 34, no. 1, 2020, pp. 1234–1241.
- [24] B. Zhao, X. Gao, J. Liu, J. Zhao, and C. Xu, “Spatiotemporal data fusion in graph convolutional networks for traffic prediction,” IEEE Access, vol. 8, pp. 76632–76641, 2020.
- [25] N. Zafar, I. U. Haq, J.-U.-R. Chughtai, and O. Shafiq, “Applying hybrid LSTM-GRU model based on heterogeneous data sources for traffic speed prediction in urban areas,” Sensors, vol. 22, no. 9, p. 3348, Apr. 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/9/3348>